TRƯỜNG ĐẠI HỌC VĂN LANG
KHOA CÔNG NGHỆ THÔNG TIN
CHUYÊN ĐỀ TỐT NGHIỆP 1 · HỌC KỲ 1, NĂM HỌC 2026 – 2027

BÁO CÁO BÀI TẬP 2

Hiện thực hoá module cốt lõi

Tiếp nhận và phân loại yêu cầu bảo hành trên hệ thống Smart CRM – Mekong Mobile

Mã luồng nghiệp vụ	**L2 – Tiếp nhận và phân loại yêu cầu bảo hành**
Track	SE (Kỹ thuật phần mềm)
Họ và tên	Bùi Quốc Triệu
MSSV	2374802010521
Lớp	261_71ITGR40203_05
Giảng viên hướng dẫn	ThS. Nguyễn Minh Tân
Ngày nộp	30/10/2026
Link GitHub repo	https://github.com/TrieuBui27/smartcrm-2374802010521-Receive-and-classify-warranty-requests
Tag nộp bài	v0.2 · https://github.com/TrieuBui27/smartcrm-2374802010521-Receive-and-classify-warranty-requests/tree/v0.2

TP. Hồ Chí Minh – năm 2026

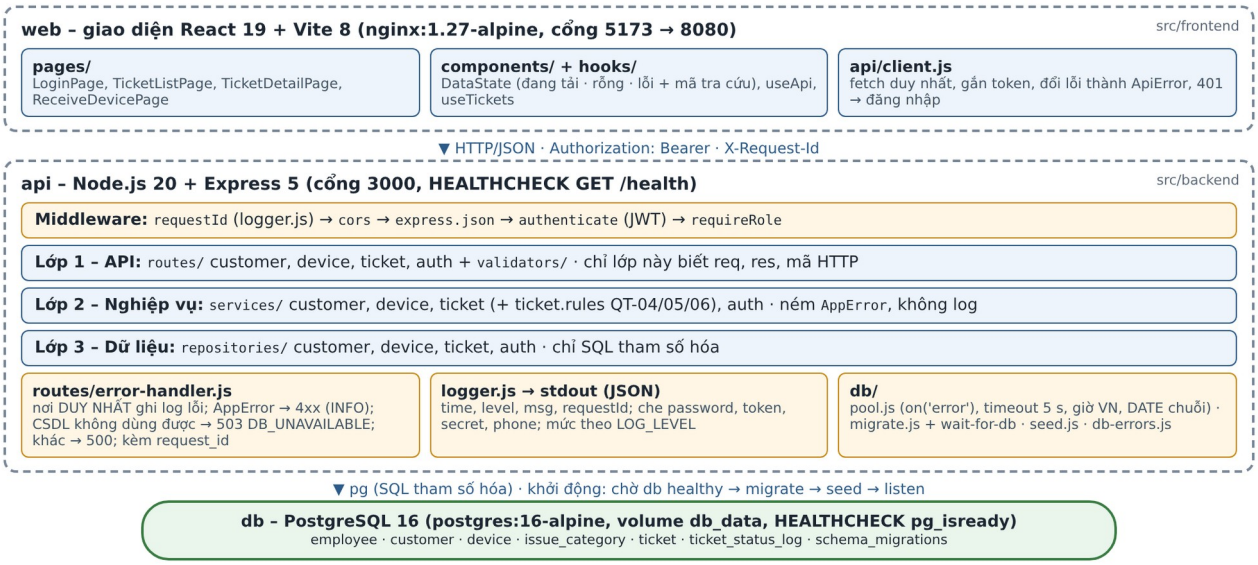
MỤC 1 – TÓM TẮT HỆ THỐNG ĐÃ LÀM

Hệ thống hỗ trợ Nhân viên tiếp nhận và Quản lý trung tâm của các trung tâm bảo hành Mekong Mobile thực hiện luồng L2: tra cứu khách theo số điện thoại, ghi nhận thiết bị, lập phiếu bảo hành kèm mô tả lỗi; hệ thống kiểm tra tình trạng bảo hành theo ngày mua, đề xuất nhóm sự cố và mức ưu tiên từ mô tả, tự sinh hạn cam kết 24/72/120 giờ (bỏ Chủ nhật); Quản lý theo dõi danh sách phiếu (đánh dấu quá hạn) và chuyển trạng thái theo vòng đời, mỗi lần chuyển ghi nhật ký. Ngăn xếp: Node.js 20 + Express 5 + PostgreSQL 16 ở backend, React 19 + Vite 8 ở giao diện, đóng gói bằng Docker Compose.

Trạng thái	Nội dung
Đã xong	• Backend ba lớp cho 4 nhóm chức năng: khách hàng (CRUD, QT-01, QT-02, QT-13, QT-15), thiết bị (CRUD, QT-03), phiếu bảo hành (tạo UC3, danh sách UC6, chi tiết, đổi trạng thái UC7, đề xuất nhóm sự cố UC4), đăng nhập JWT và phân quyền theo vai trò, theo trung tâm (QT-14). • Bốn màn hình React gọi API thật: đăng nhập, danh sách phiếu (lọc, phân trang), chi tiết phiếu (đổi trạng thái), tiếp nhận thiết bị; đủ ba trạng thái đang tải, rỗng, lỗi. • 70 kiểm thử tự động (`npm test`) thuộc ba loại luồng đúng, biên, dữ liệu sai; test viết trước code cho UC3 6a, 8a, UC7 3a, 4b và CRUD thiết bị. • 7 migration SQL, seed từ bộ dữ liệu học phần chạy lại không nhân đôi; Docker Compose dựng db, api, web bằng hai lệnh. • Log JSON có requestId, một middleware lỗi duy nhất; CSDL tắt thì API trả 503 kèm mã tra cứu, backend không sập (BUG-002). Bug Log 3 mục.
Chưa xong	• Màn hình tạo hồ sơ khách mới và ghi nhận thiết bị mới (đã có API, chưa có giao diện). • Trang tổng quan của Quản lý, báo cáo, quản lý tài khoản (ngoài phạm vi L2).

Số liệu trên dữ liệu mẫu (trích từ bộ dữ liệu Mekong Mobile của học phần, xem TD-10, TD-12): 6 nhóm sự cố, 12 tài khoản nhân viên (6 trung tâm × 2 vai trò), 139 khách hàng (11 dòng số điện thoại không chuẩn hóa được bị bỏ qua), 25 thiết bị, 25 phiếu bảo hành ở đủ các trạng thái, 119 dòng nhật ký trạng thái; trung tâm 4 có sẵn phiếu quá hạn.

MỤC 2 – KIẾN TRÚC ĐÃ HIỆN THỰC



Hình 2.1. Thành phần thực tế trong mã nguồn và ba dịch vụ Docker Compose (web, api, db)

Lớp	Thư mục	Không được làm (giữ ranh giới lớp)
Lớp 1 – API	`src/backend/routes/`, `validators/`, `middleware/`	Chứa quy tắc nghiệp vụ, gọi SQL. Chỉ lớp này đọc `req`, ghi `res`, đổi mã lỗi sang mã HTTP

Lớp	Thư mục	Không được làm (giữ ranh giới lớp)
Lớp 2 – Nghiệp vụ	`src/backend/services/`	Dùng `req`, `res`, mã trạng thái HTTP, viết SQL; không ghi log, chỉ ném `AppError` có mã
Lớp 3 – Dữ liệu	`src/backend/repositories/`	Chứa quy tắc nghiệp vụ; chỉ SQL tham số hóa ($1, $2…)
Hạ tầng	`src/backend/db/`, `logger.js`, `routes/error-handler.js`	Một pool kết nối dùng chung; log chỉ ghi ở middleware lỗi và khi yêu cầu kết thúc
Giao diện	`src/frontend/src/api/`, `hooks/`, `components/`, `pages/`	Gọi `fetch` ngoài `api/client.js`; ghi cứng địa chỉ backend (lấy từ `VITE_API_BASE_URL`)

Docker Compose khởi động theo thứ tự có kiểm tra sức khỏe: `db` healthy (pg_isready) → `api` chạy migrate, seed rồi mở cổng, healthy khi `GET /health` truy vấn được CSDL → `web`. So với kiến trúc BT1, các lớp giữ nguyên; phần thêm mới là middleware xác thực, logger, bộ xử lý lỗi và lớp đóng gói Docker (TD-13, TD-18, TD-19).

MỤC 3 – MODULE CỐT LÕI

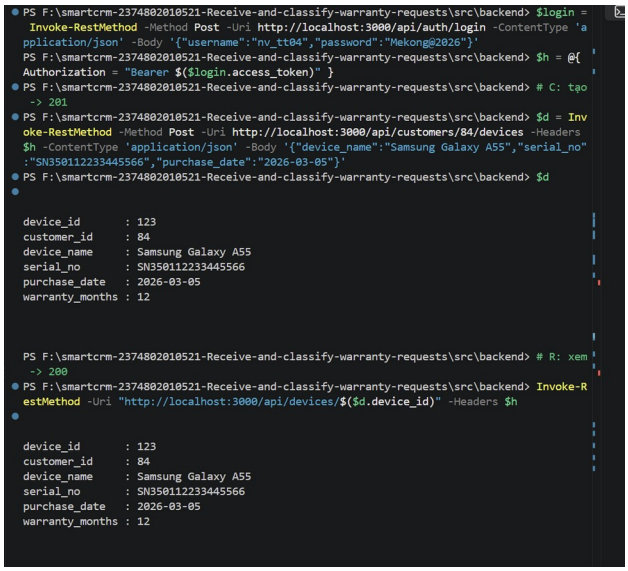
Quy tắc	Nội dung đã hiện thực	Ở đâu trong mã
QT-02, QT-15	Chuẩn hóa số điện thoại về 0xxxxxxxxx; che số `097****148` với Nhân viên tiếp nhận	`services/phone.js`, `ticket.contract.js`
QT-03	Serial/IMEI không trùng (409 `SERIAL_EXISTS`); mỗi thiết bị tối đa một phiếu chưa đóng (409 `DEVICE_HAS_OPEN_TICKET`, chỉ mục duy nhất `ux_ticket_device_open` chặn cả khi hai yêu cầu cùng lúc)	`device.service.js`, `ticket.service.js create`, `migrations/005`
QT-04	Hạn cam kết CAO 24 giờ, TRUNG_BINH 72 giờ, THAP 120 giờ, bỏ Chủ nhật; luôn do hệ thống tính	`ticket.rules.js computeDueDate`
QT-05	Còn bảo hành nếu ngày tiếp nhận chưa quá ngày mua + số tháng; thiếu ngày mua thì "chưa xác minh"	`ticket.rules.js checkWarranty`
QT-06	Vòng đời một chiều, không quay lại; Nhân viên chỉ đóng hoặc hủy (422 `INVALID_TRANSITION`, 403)	`ticket.rules.js`, `ticket.repository.js updateStatusIfCurrent`
QT-14	Chỉ xem, sửa phiếu của trung tâm mình (403)	`ticket.service.js`
Thiết bị	Không xóa thiết bị đã có phiếu (409 `DEVICE_HAS_TICKETS`) để giữ lịch sử bảo hành	`device.service.js remove`

Tệp service không import Express, không đọc `req`, không đặt mã HTTP: lỗi nghiệp vụ được ném dưới dạng `AppError(code, message)` và `routes/error-handler.js` đổi mã sang HTTP. Hai thực thể đủ CRUD qua API thật: **khách hàng** (`/api/customers`, buổi 8) và **thiết bị** (`/api/customers/{id}/devices`, `/api/devices/{id}`, TD-17). Phiếu bảo hành có tạo, đọc, sửa trạng thái; không xóa vật lý mà chuyển sang Đã hủy theo QT-06.

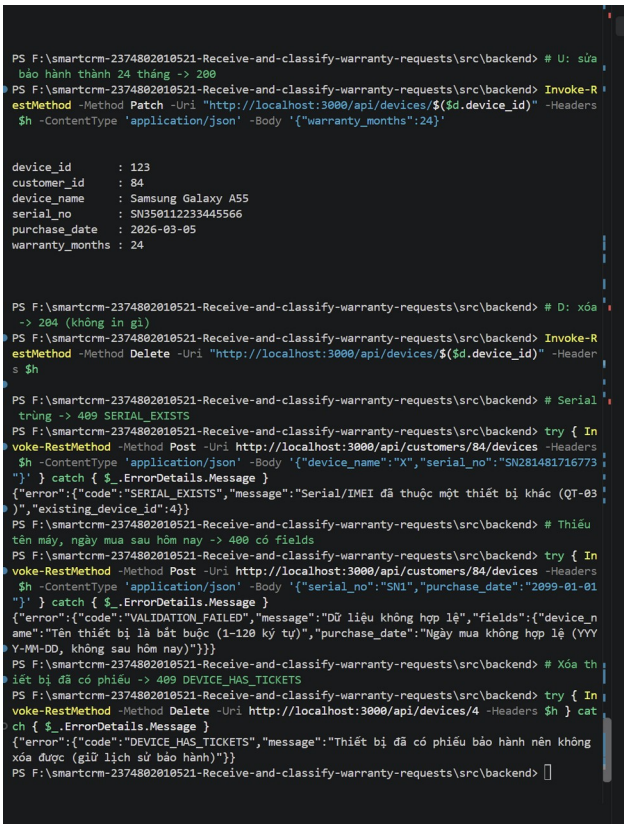
bash — gọi API thật: quy tắc nghiệp vụ

```
# Đăng nhập Nhân viên tiếp nhận TT04, lưu token vào $TOKEN (Quản lý TT04: $QL)
$ curl -s -X POST localhost:3000/api/auth/login -H 'Content-Type: application/json' -d '{"username":"nv_tt04","password":"Mekong@2026"}' | head -c 1
20
{"access_token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjo4LCJyb2xlIjoiTkhBTl9WSUVOX1RJRVBfTkhBTiIsImNlbnRlcl
# UC1 tra cứu theo số có dấu cách: chuẩn hóa QT-02, chỉ số theo QT-15
$ curl -s 'localhost:3000/api/customers?phone=0974%20799%20148' -H "Authorization: Bearer $TOKEN"
{"customer_id":84,"full_name":"Phan Quang Nam","phone":"097****148","email":"nam720@gmail.com","address":"120 Lê Lợi, Thốt Nốt","is_active":true,"cr
eated_at":"2026-10-29T17:36:07.995Z","devices":[{"device_id":1,"device_name":"TECNO Lite2","serial_no":"SN281481716773","purchase_date":"2024-10-1
3","warranty_months":12}]}
# UC3 6a: thiết bị 3 đang có phiếu chưa đóng -> 409 kèm mã phiếu cũ
$ curl -s -w ' HTTP %{http_code}' -X POST localhost:3000/api/tickets -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' -d '{"cus
tomer_id":44,"device_id":3,"issue_desc":"man hinh bi soc doc sau khi roi"}'
{"error":{"code":"DEVICE_HAS_OPEN_TICKET","message":"Thiết bị đang có phiếu bảo hành chưa đóng","open_ticket_code":"BH-000231/2025"}} HTTP 409
# UC7 3a: phiếu Đã đóng không được quay lại Đang xử lý -> 422
$ curl -s -w ' HTTP %{http_code}' -X PATCH localhost:3000/api/tickets/1112/status -H "Authorization: Bearer $QL" -H 'Content-Type: application/json'
-d '{"to_status":"DANG_XU_LY"}'
{"error":{"code":"INVALID_TRANSITION","message":"Không thể chuyển từ DA_DONG sang DANG_XU_LY"}} HTTP 422
# QT-14: xem phiếu của trung tâm khác -> 403
$ curl -s -w ' HTTP %{http_code}' localhost:3000/api/tickets/101 -H "Authorization: Bearer $QL"
{"error":{"code":"FORBIDDEN","message":"Phiếu thuộc trung tâm khác"}} HTTP 403
```

Hình 3.1. Quy tắc nghiệp vụ khi gọi API thật (Nhân viên và Quản lý trung tâm 4): QT-02/15, UC3 6a 409, UC7 3a 422, QT-14 403



Hình 3.2. CRUD thiết bị qua API thật (chụp trên máy em): tạo 201, đọc 200



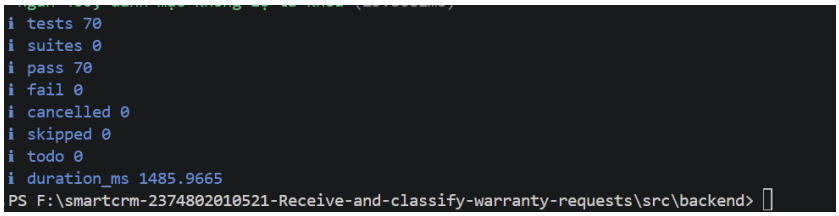
Hình 3.3. Sửa 200, xóa 204, serial trùng 409, dữ liệu sai 400 có fields, xóa thiết bị có phiếu 409

MỤC 4 – BỘ KIỂM THỬ

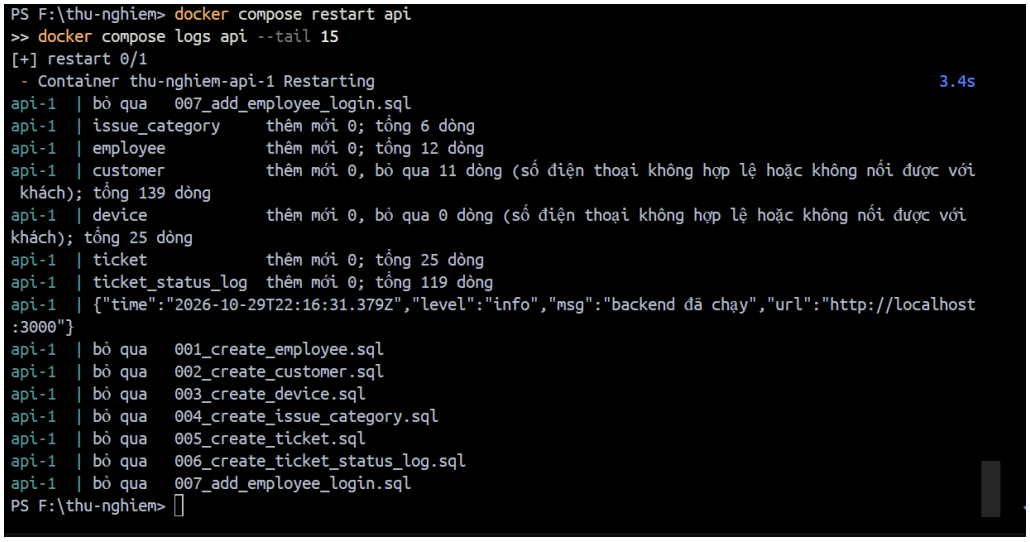
Chạy bằng một lệnh `npm test` trong `src/backend` (ghi trong README mục 5), dùng `node:test` có sẵn của Node.js. Test dựng dữ liệu riêng: repository giả trong bộ nhớ cho test service và API (khởi động Express thật ở cổng ngẫu nhiên), test repository trên PostgreSQL thật tự tạo rồi xóa dữ liệu của nó và tự bỏ qua nếu chưa có CSDL. Tổng **70 test PASS**, gồm cả ba loại bắt buộc:

Loại	Trường hợp tiêu biểu	Tệp
1. Luồng đúng	Tạo phiếu: trạng thái MOI, đề xuất nhóm PIN, hạn 72 giờ; CRUD thiết bị 201 → 200 → 200 → 204 → 404; CSDL lên sau 2 lần từ chối thì migrate chạy tiếp	`ticket.service.test.js`, `device.test.js`, `db.wait.test.js`

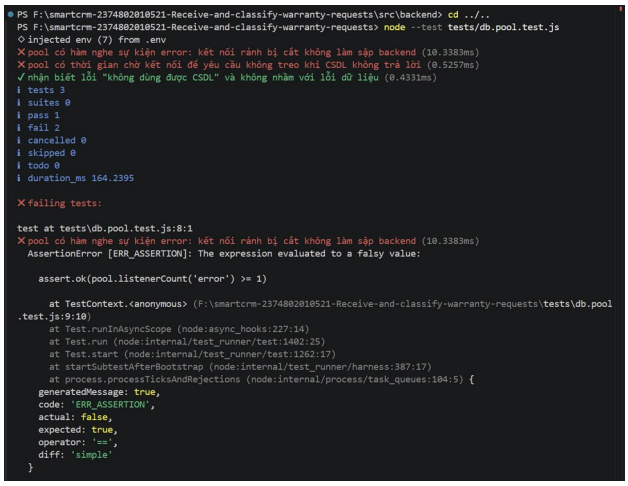
Loại	Trường hợp tiêu biểu	Tệp
2. Giá trị biên	QT-04 hạn đi qua Chủ nhật; QT-05 đúng ngày hết hạn vẫn còn bảo hành; mô tả đúng 10 và 2000 ký tự; bảo hành 1 và 60 tháng, ngày mua đúng hôm nay; UC3 8a và UC7 4b hai yêu cầu cùng lúc	`ticket.rules.test.js`, `ticket.create.validator.test.js`, `device.test.js`, `ticket.repository.test.js`
3. Dữ liệu sai	Số điện thoại không chuẩn hóa được; body không phải JSON 400; thiếu tên máy, ngày mua sau hôm nay 400 có fields; sai mật khẩu CSDL báo ngay; token sai 401; trung tâm khác 403	`customer.service.test.js`, `error-handler.test.js`, `device.test.js`, `ticket.api.test.js`
Hồi quy lỗi	BUG-002 pool có hàm nghe `error`, CSDL tắt trả 503; BUG-003 cột DATE đúng `YYYY-MM-DD` (FAIL trước khi sửa, PASS sau)	`db.pool.test.js`, `error-handler.test.js`



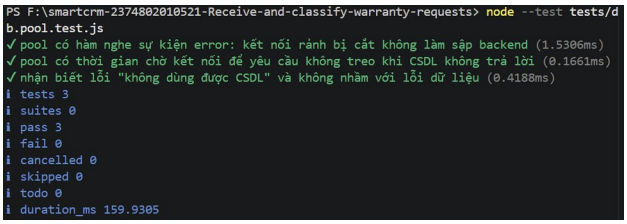
Hình 4.1. npm test chụp trên máy em: 70/70 PASS (tests 70, pass 70, fail 0, skipped 0)



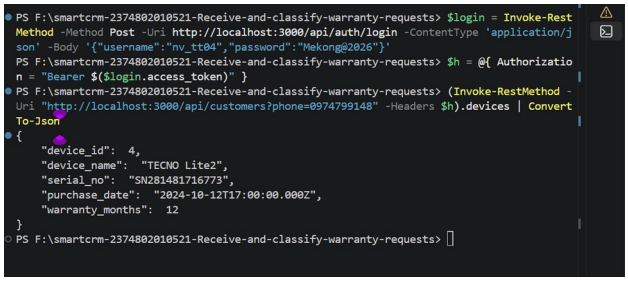
Hình 4.2. Khởi động lại api trên bản clone sạch: bỏ qua 7 migration, "thêm mới 0" ở mọi bảng, tổng dòng giữ nguyên 6/12/139/25/25/119 (C5)



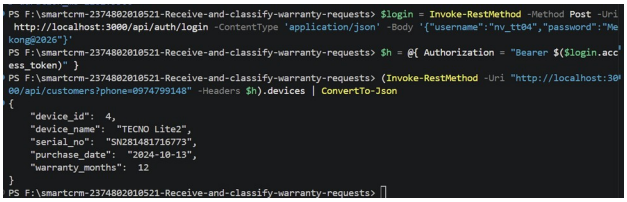
Hình 4.3. BUG-002, bước 4: test hồi quy FAIL trước khi sửa (pool chưa có hàm nghe error)



Hình 4.4. Cùng test PASS sau khi sửa db/pool.js



Hình 4.5. BUG-003: tái lập ngày mua lùi một ngày khi máy giờ Việt Nam

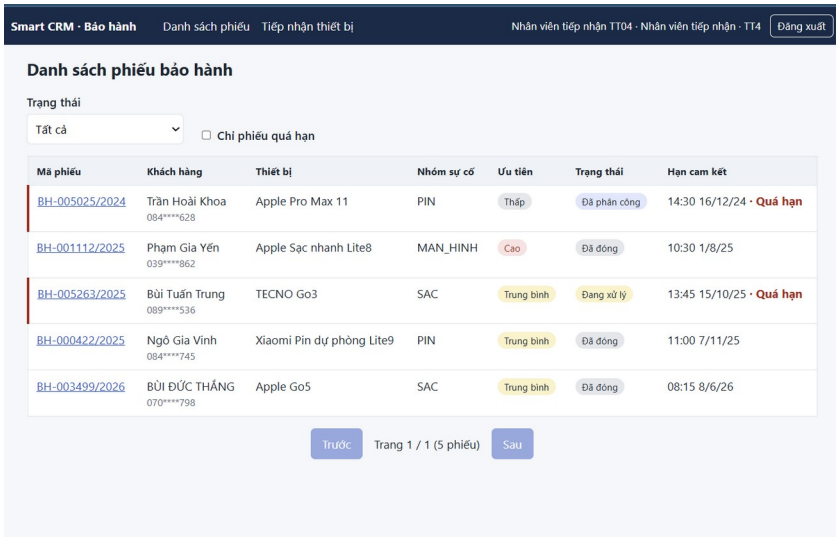


Hình 4.6. Sau khi sửa: ngày mua đúng 2024-10-13

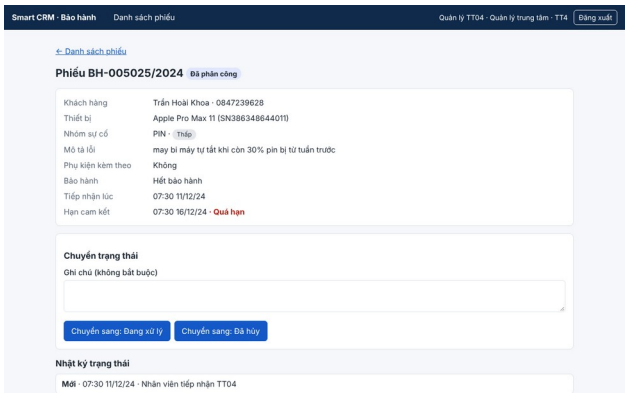
Ba lỗi đã gỡ theo bốn bước tái lập, khoanh vùng, sửa nguyên nhân, xác nhận được ghi đủ bảy dòng trong `docs/bug-log.md` kèm link commit: BUG-001 Major (TIMESTAMP lệch 7 giờ), BUG-002 Critical (backend tắt hẳn khi PostgreSQL tắt), BUG-003 Minor (ngày mua lùi một ngày trên máy giờ Việt Nam).

MỤC 5 – MODULE MỞ RỘNG VÀ TRÌNH BÀY KẾT QUẢ

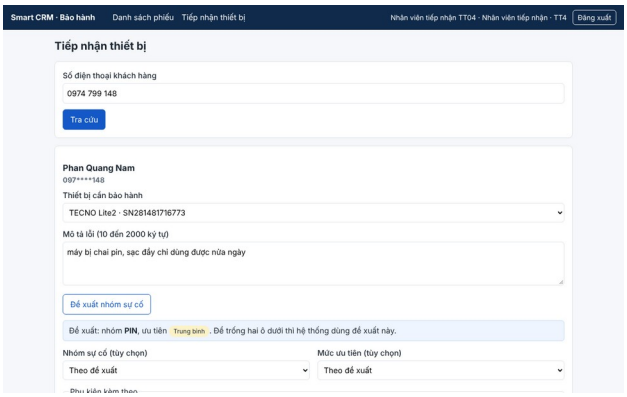
Giao diện React theo wireframe BT1, lấy dữ liệu từ API thật: mọi lời gọi đi qua `src/frontend/src/api/client.js` (gắn token, đổi lỗi thành `ApiError`, nhận 401 thì về trang đăng nhập); ba trạng thái dùng chung component `DataState.jsx`. Đăng nhập bằng 12 tài khoản mẫu, mỗi tài khoản chỉ thấy phiếu của trung tâm mình.



Hình 5.1. Màn hình 1 – Danh sách phiếu trung tâm 4, chụp trên bản Docker v0.2 của em (Quản lý ql_tt04): trạng thái, hạn cam kết, phiếu quá hạn viền đỏ, phân trang



Hình 5.2. Màn hình 2 – Chi tiết phiếu: nhật ký trạng thái, chỉ hiện nút chuyển hợp lệ (allowed_transitions)

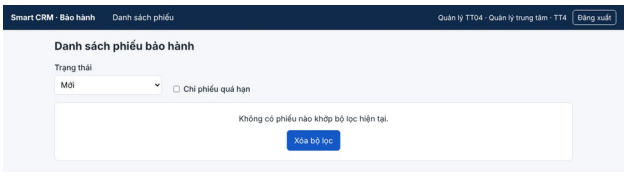


Hình 5.3. Màn hình 3 – Tiếp nhận thiết bị: tra cứu khách (số đã che), đề xuất nhóm PIN, ưu tiên Trung bình

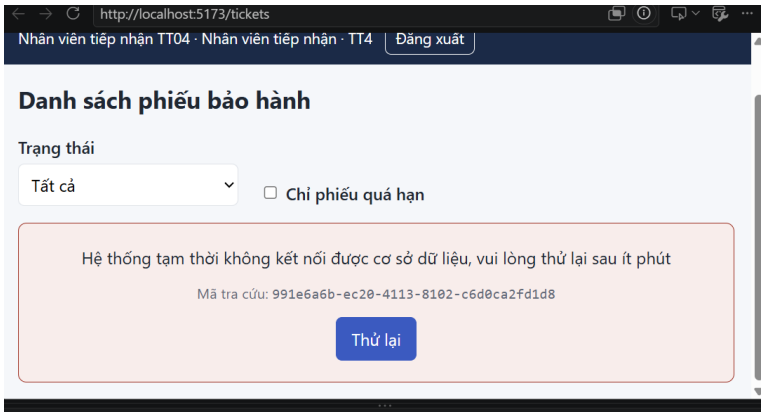
Ba trạng thái



Hình 5.4. Đang tải: "Đang tải dữ liệu…" trong lúc chờ API



Hình 5.5. Rỗng do bộ lọc: nói rõ không có phiếu khớp, có nút Xóa bộ lọc



Hình 5.6. Lỗi (chụp trên máy em): tắt PostgreSQL rồi tải lại, API trả 503 kèm mã tra cứu, có nút Thử lại; bật lại CSDL bấm Thử lại là dùng được

Suy giảm có kiểm soát: khi tắt CSDL, backend không sập (BUG-002 đã sửa), trả 503 `DB_UNAVAILABLE` với thông báo dễ hiểu; chi tiết kỹ thuật (mã lỗi, stack) chỉ nằm trong log JSON, tìm được bằng đúng mã tra cứu. Không bao giờ trả câu SQL hay tên bảng cho người dùng.

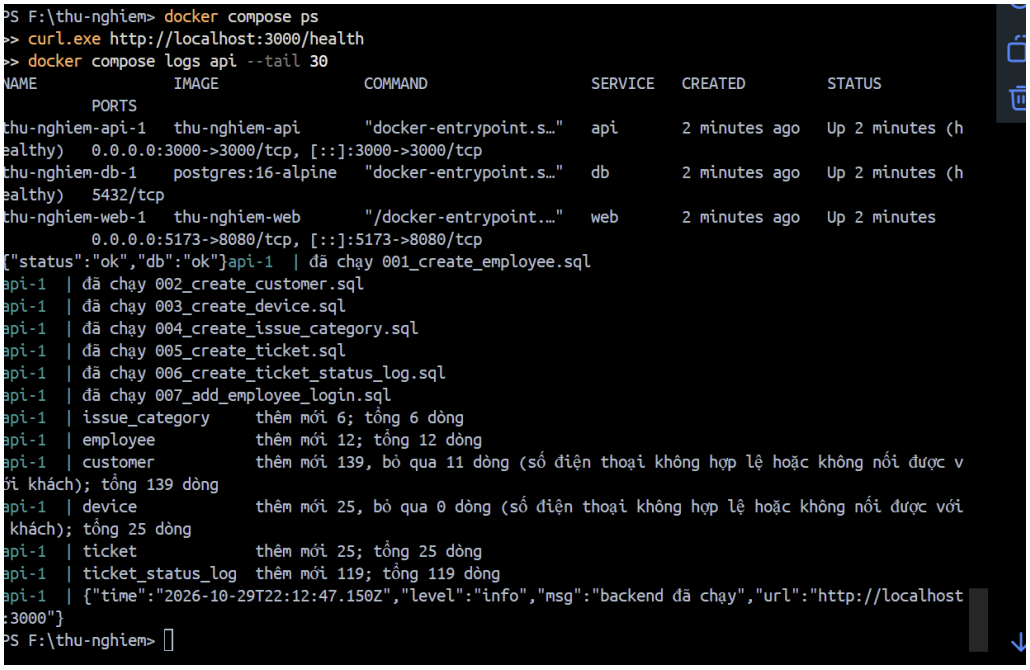
MỤC 6 – CÁCH DỰNG LẠI HỆ THỐNG

Cách 1 – Docker Compose, hai lệnh (README mục 5, chỉ cần Docker Desktop; cổng 3000 và 5173 phải trống):

```
$ git clone --branch v0.2 https://github.com/TrieuBui27/smartcrm-2374802010521-Receive-and-classify-warranty-requests.git thu-nghiem
$ cd thu-nghiem
$ cp .env.example .env              # PowerShell: Copy-Item .env.example .env
$ docker compose up -d --build
$ docker compose ps                 # db, api "healthy"; web "running"
$ curl http://localhost:3000/health # {"status":"ok","db":"ok"}
# Mở http://localhost:5173, đăng nhập nv_tt04 hoặc ql_tt04, mật khẩu Mekong@2026
```

Cách 2 – Node.js 20.19+ và PostgreSQL 14+ cài trên máy (README mục 5, cách 2):

```
$ cp .env.example .env              # sửa DB_USER, DB_PASSWORD, SECRET_KEY
$ psql -U postgres -c "CREATE DATABASE smartcrm;"
$ cd src/backend && npm ci
$ npm run db:migrate && npm run db:seed   # chạy lại không lỗi, không nhân đôi
$ npm test                                # 70 PASS
$ npm start                               # http://localhost:3000
$ cd ../frontend && cp .env.example .env && npm ci && npm run dev   # http://localhost:5173
```



Hình 6.1. Phép thử thư mục trống (chụp trên máy em, tag v0.2): db và api healthy, web chạy, /health ok, migrate và seed tự chạy

Tệp khóa phiên bản `package-lock.json` đã commit cho cả backend và frontend (README dùng `npm ci`); ảnh Docker ghim phiên bản (`node:20-alpine`, `postgres:16-alpine`, `nginx:1.27-alpine`), container chạy bằng người dùng không đặc quyền. `.env.example` liệt kê đủ tên biến với giá trị mẫu; `.env` thật nằm trong `.gitignore`, rà `grep -rniE "password|api_key|secret" src/` không có giá trị thật. Đã làm phép thử: clone vào thư mục mới → làm đúng README → hệ thống lên được bằng Docker Compose (cách 1).

MỤC 7 – THAY ĐỔI SO VỚI THIẾT KẾ BÀI TẬP 1

Ghi đầy đủ theo bốn dòng (phản hồi → quyết định → lý do → ảnh hưởng) trong `docs/thay-doi-thiet-ke.md`; TD-01 đến TD-08 là xử lý phản hồi BT1. Các thay đổi khi hiện thực BT2:

Mã	Thay đổi	Vì sao
TD-09	Dùng thư viện `pg` và migration SQL đánh số thay cho Prisma	Giữ đúng DDL BT1, viết SQL trực tiếp, ít phụ thuộc
TD-10, 12	Seed trích từ bộ dữ liệu học phần; ngày mua tự sinh; quy ước lưu TIMESTAMP giờ Việt Nam	Bộ dữ liệu không có ngày mua; tránh lệch 7 giờ (BUG-001)
TD-11	`test/` đổi thành `tests/`; `migrations/`, `seeds/` ở gốc repo	Theo cấu trúc chuẩn của học phần
TD-13	Đăng nhập tối thiểu bằng JWT (BT1 xếp WON'T)	Cần danh tính để áp QT-14, QT-15 và vai trò
TD-14	Chi tiết phiếu trả thêm `allowed_transitions`	Một nguồn sự thật cho vòng đời QT-06, giao diện không sao chép luật

Mã	Thay đổi	Vì sao
TD-15, 17	Buổi 9–10 chưa làm API thiết bị; buổi 11–12 làm đủ CRUD thiết bị, chưa có màn hình	Tiêu chí S1 cần hai thực thể đủ CRUD
TD-16	CORS một nguồn; tài khoản mẫu chung mật khẩu thử nghiệm	Chạy cục bộ, chỉ để thử nghiệm
TD-18	Log JSON có requestId, một middleware lỗi, 503 `DB_UNAVAILABLE`, `GET /health`	Yêu cầu buổi 11; phép thử suy giảm làm lộ BUG-002, BUG-003
TD-19	Đóng gói Docker Compose db, api, web	Phép thử thư mục trống (C1), chạy được bằng hai lệnh

PHỤ LỤC – BẢNG KHAI BÁO SỬ DỤNG CÔNG CỤ AI (BÀI TẬP 2)

Công cụ	Dùng vào việc gì	Áp dụng ở tệp / hàm nào	Em đã kiểm chứng thế nào
Claude	Gợi ý migration 6 bảng, seed, các lớp module khách hàng và 8 unit test (buổi 8)	`migrations/001–006`, `db/seed.js`, `customer.*.js`, `tests/phone.test.js`, `customer.service.test.js`	Tự gõ và chạy: migrate, seed hai lần không nhân đôi; gọi API bằng curl ra 200/400/404; đối chiếu migration với DDL BT1; sửa `customer_id` trả về dạng chuỗi thành số
Claude	Gợi ý backend phiếu bảo hành, đăng nhập JWT, phân quyền, test viết trước UC3 6a, 8a, UC7 3a, 4b (buổi 9–10)	`ticket.rules.js`, `ticket.service.js`, `ticket.repository.js`, `auth.service.js`, `middleware/`, `migrations/007`	Chạy test đỏ trước rồi mới code; npm test 47 pass; gọi API ra đúng 200, 400, 401, 403, 404, 409, 422; kiểm phiếu trung tâm khác bị 403, số điện thoại bị che theo vai trò
Claude	Gợi ý giao diện React 4 màn hình, lớp gọi API, ba trạng thái (buổi 9–10)	`src/frontend/src/`	Chạy giao diện với dữ liệu thật; thử đăng nhập sai, lọc rỗng, tắt backend, khung 375px; kiểm không còn `fetch` ngoài `api/client.js`
Claude	Buổi 11–12: logger JSON, middleware lỗi và 503, cách gỡ BUG-002, BUG-003, CRUD thiết bị (test trước), Dockerfile và docker-compose	`logger.js`, `routes/error-handler.js`, `db/pool.js`, `db/db-errors.js`, `db/wait-for-db.js`, `device.*.js`, `Dockerfile`, `docker-compose.yml`	Tự tái lập hai lỗi (tắt PostgreSQL khi backend chạy; ngày mua khi máy đặt giờ Việt Nam), thấy test hồi quy FAIL trước, PASS sau; npm test 70 pass; gọi API thiết bị ra 201/400/404/409/204; chạy Docker từ bản clone mới; đọc log kiểm không lộ mật khẩu, token, số điện thoại
Claude	Soạn tài liệu: README, hợp đồng API, TD-09 đến TD-19, `docs/bug-log.md`, bản nháp báo cáo BT2	`README.md`, `docs/`	Đọc lại từng mục, đối chiếu endpoint và mã lỗi với code; làm lại đúng các lệnh README trên bản clone mới; bấm thử từng link commit trong bug-log
Không dùng	Chọn luồng L2, phạm vi (bỏ kỹ thuật viên, L4/L5/L8/L10), cách chạy thử và chụp ảnh kết quả	Máy cá nhân, repo GitHub	Không áp dụng

Tôi xác nhận đã đọc, hiểu và chịu trách nhiệm về toàn bộ nội dung nộp. Bùi Quốc Triệu – 2374802010521 – 30/10/2026.